Some title of your project

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Suneet Singla	102303615
2	Bipasha	102303388

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2026

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Introduction to Problem Statement 2

2 Literature Review **3**

3 Research Gaps **4**

4 Problem Formulation **5**

5 Objectives **6**

6 Methodology **7**

6.1 Scheduling Algorithms 8

6.1.1 Load-Aware Scheduling (Proposed) 9

6.2 Equation 10

7 Datasets **11**

8 Results and Discussion **12**

8.1 Observations 12

8.2 Detailed Analysis 13

9 Conclusion **15**

1 Introduction

Parallel computing enables multiple computations to execute simultaneously, improving performance over sequential execution. With the rise of multi-core processors, efficient scheduling of tasks has become essential to fully utilize available resources.

This project presents a parallel task scheduling simulator that models tasks using a Directed Acyclic Graph (DAG) and evaluates different scheduling algorithms based on performance metrics such as makespan, utilization, load imbalance, and speedup.

Parallel computing has become a fundamental paradigm in modern computing systems due to the increasing demand for high-performance applications such as scientific simulations, big data analytics, artificial intelligence, and real-time processing systems.

With the evolution of multi-core and many-core architectures, efficient utilization of computational resources has become a critical challenge. Task scheduling plays a central role in ensuring that workloads are distributed effectively across processors while minimizing execution time and maximizing resource utilization.

In real-world systems, tasks often exhibit dependencies, meaning certain tasks cannot begin execution until others have completed. These dependencies significantly constrain parallel execution and make scheduling more complex. Therefore, intelligent scheduling strategies are required to balance competing objectives such as minimizing makespan, reducing idle time, and maintaining load balance.

This project aims to address these challenges by simulating a DAG-based scheduling environment and evaluating multiple classical and heuristic scheduling algorithms. The study focuses not only on performance comparison but also on understanding how system parameters such as task size, processor count, and dependency density influence scheduling efficiency.

1.1 Background

In parallel systems, tasks are distributed across multiple processors for execution. However, dependencies between tasks restrict execution order and limit achievable parallelism.

These dependencies are modeled using a Directed Acyclic Graph (DAG), where nodes represent tasks and edges represent precedence constraints.

Efficient scheduling ensures that processors remain utilized while respecting these constraints.

1.2 Introduction to Problem Statement

Task scheduling in parallel systems is a complex problem due to task dependencies and varying execution times. Poor scheduling leads to increased makespan, processor idle time, and load imbalance.

The objective of this project is to simulate task scheduling using multiple algorithms and analyze their effectiveness in improving system performance.

2 Literature Review

Task scheduling in parallel and distributed systems has been extensively studied due to its critical impact on system performance. Various algorithms have been proposed to optimize metrics such as makespan, resource utilization, and load balancing.

One of the most widely used approaches is the HEFT (Heterogeneous Earliest Finish Time) algorithm proposed by Topcuoglu et al. [1]. This algorithm ranks tasks based on their upward rank and assigns them to processors that minimize earliest completion time. While HEFT achieves good performance, it assumes accurate estimation of execution and communication costs, which may not always be realistic.

Braun et al. [2] conducted a comprehensive comparison of eleven static scheduling heuristics, including Min-Min and Max-Min algorithms. Their study highlighted that Min-Min performs well for smaller tasks by prioritizing tasks with minimum execution time, whereas Max-Min attempts to reduce starvation by prioritizing larger tasks. However, both approaches suffer from load imbalance under certain workload distributions.

Kwok and Ahmad [3] provided a detailed survey of static scheduling algorithms for DAG-based task graphs. Their work emphasized the importance of considering precedence constraints and highlighted the computational complexity associated with optimal scheduling.

Classical scheduling theory, as discussed by Graham [4] and Coffman [5], provides foundational insights into multiprocessor scheduling and its inherent challenges. These works establish that optimal scheduling is NP-hard, necessitating the use of heuristic and approximation-based approaches.

Blazewicz et al. [6] and Sinnen [7] further explored scheduling techniques in parallel systems, focusing on both theoretical models and practical implementations. Their studies emphasize the trade-off between scheduling optimality and computational overhead.

Recent research has also explored dynamic scheduling techniques that adapt to runtime conditions. Ahmad and Kwok [8] investigated task duplication strategies to reduce communication delays, while Li [9] proposed methods to minimize processor idle time by improving task allocation strategies.

Despite these advancements, existing approaches often focus on optimizing a single performance metric, such as makespan, while neglecting load balancing and utilization.

3 Research Gaps

Despite extensive research in parallel task scheduling, several challenges remain unaddressed.

First, many studies focus on evaluating algorithms under limited configurations, often ignoring variations in task size, processor count, and dependency density. This limits the generalizability of results.

Second, visualization of performance metrics is often underutilized. Graphical analysis provides intuitive insights into algorithm behavior, yet many works rely solely on numerical comparisons.

Third, load balancing is frequently overlooked as a primary optimization objective. While minimizing makespan is important, uneven workload distribution can lead to inefficient resource utilization.

Additionally, most existing approaches assume static environments where task characteristics are known in advance. Real-world systems, however, are dynamic and require adaptive scheduling strategies.

This project aims to bridge these gaps by:

- Evaluating multiple algorithms under diverse configurations
- Incorporating graphical performance analysis
- Introducing a load-aware scheduling strategy
- Analyzing trade-offs between different performance metrics

4 Problem Formulation

Let $T = \{t_1, t_2, \ldots, t_n\}$ be a set of tasks.

Each task has:

- Execution time e_i
- Dependency set D_i

The objective is to schedule these tasks on P processors such that:

- Makespan is minimized
- Dependencies are satisfied
- Load is balanced across processors

Since optimal scheduling is NP-hard, heuristic-based algorithms are used.

Constraints and Assumptions

The following assumptions are considered in this model:

- All processors are homogeneous in nature
- Each processor can execute only one task at a time
- Communication cost between tasks is ignored
- Tasks are non-preemptive once assigned

Objective Optimization

The scheduling problem aims to minimize the makespan while maintaining load balance:

$$\min \ M = \max(C_i)$$

Additionally, the system aims to minimize load imbalance:

$$LI = \frac{\max(load) - \min(load)}{\text{average load}}$$

5 Objectives

The primary objective of this project is to design and analyze an efficient task scheduling system for parallel computing environments using Directed Acyclic Graph (DAG) models.

The specific objectives of this work are as follows:

- To design and implement a parallel task scheduling simulator capable of handling task dependencies using DAG representation.
- To study and implement multiple classical scheduling algorithms such as FCFS, Round Robin, Min-Min, and Max-Min.
- To develop a custom Load-Aware scheduling algorithm that dynamically assigns tasks based on processor workload.
- To evaluate and compare the performance of different scheduling algorithms using standard metrics such as makespan, utilization, speedup, and load imbalance.
- To analyze the impact of varying system parameters, including task size, processor count, and dependency density, on scheduling performance.
- To ensure efficient utilization of computational resources by minimizing processor idle time.
- To study the trade-offs between different scheduling strategies in terms of performance and computational complexity.
- To visualize and interpret experimental results using graphical representations for better understanding.
- To investigate the scalability of scheduling algorithms as the number of tasks and processors increases.
- To identify limitations of existing scheduling techniques and propose improvements through load-aware strategies.
- To provide a flexible and extensible simulation framework that can be adapted for future research in parallel and distributed computing.

6 Methodology

The proposed system simulates a parallel task scheduling environment using a Directed Acyclic Graph (DAG). The workflow consists of the following steps:

- Task generation with random execution times
- Creation of task dependencies forming a DAG
- Selection of scheduling algorithm
- Assignment of tasks to processors
- Parallel execution using OpenMP
- Calculation of performance metrics
- Storage of results in CSV format
- Visualization using graphs

System Architecture

The system is designed as a modular simulation framework consisting of the following components:

- **Task Generator:** Generates tasks with random execution times and dependencies.
- **DAG Constructor:** Builds a Directed Acyclic Graph representing task relationships.
- **Scheduler Module:** Implements various scheduling algorithms.
- **Execution Engine:** Simulates parallel execution using OpenMP.
- **Metrics Calculator:** Computes performance metrics such as makespan, utilization, speedup, and load imbalance.
- **Visualization Module:** Generates graphs for analysis.

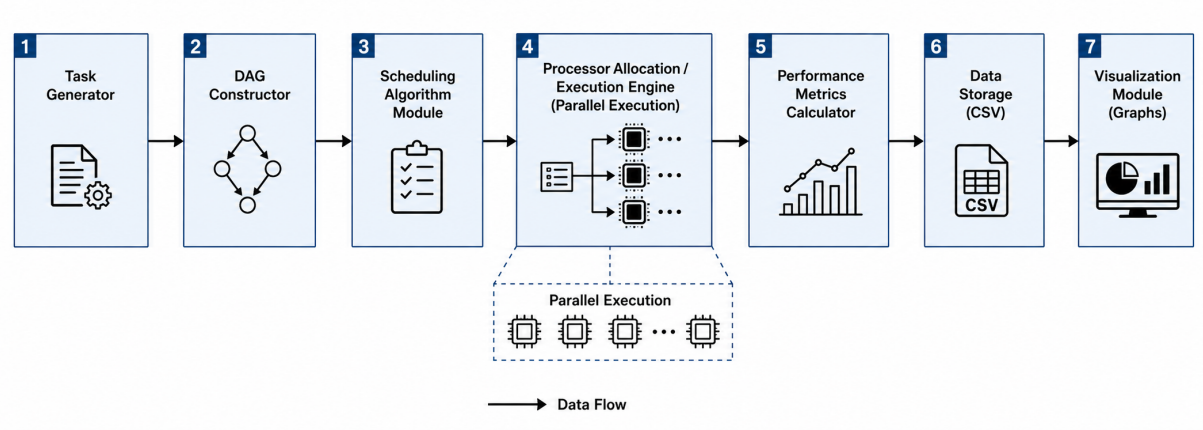


Figure 1: Example of a layered Directed Acyclic Graph (DAG) representing task dependencies

Working Principle

The simulation begins with task generation, followed by DAG construction. Scheduling algorithms are then applied to assign tasks to processors. Tasks are executed in parallel while respecting dependencies. Finally, performance metrics are computed and stored for analysis.

This modular design allows easy integration of additional scheduling algorithms and scalability for larger experiments.

6.1 Scheduling Algorithms

The following algorithms are implemented:

- FCFS (First Come First Serve)
- Round Robin
- Min-Min
- Max-Min
- Load-Aware (custom algorithm)

Algorithm 1 Parallel Task Scheduling Simulator

```
Generate tasks with execution time and dependencies
Construct DAG
for each scheduling algorithm do
    Initialize processors
    while tasks remain do
        Select ready tasks (no pending dependencies)
        Assign tasks to processors
        Execute tasks in parallel
    end while
    Compute metrics (makespan, utilization, speedup)
end for
```

6.1.1 Load-Aware Scheduling (Proposed)

The Load-Aware scheduling algorithm is a custom heuristic designed to improve load balancing across processors.

The algorithm assigns each task to the processor with the minimum current workload. At each step, ready tasks are selected and assigned dynamically based on processor load.

Steps:

- Identify ready tasks
- Calculate current load on each processor
- Assign task to least loaded processor
- Update processor load

Advantages:

- Minimizes load imbalance
- Improves processor utilization
- Reduces makespan

Limitation: It is a greedy approach and does not consider future task dependencies or critical path.

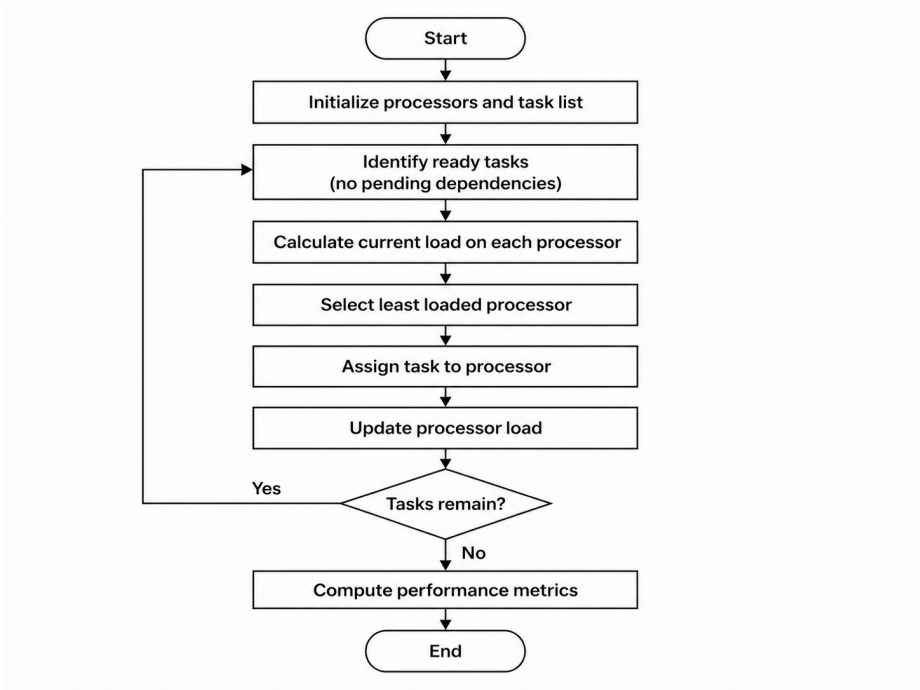


Figure 2: Flowchart of the proposed Load-Aware scheduling algorithm

6.2 Equation

The following performance metrics are used:

Makespan:

$$M = \max_i(finish_time_i)$$

Utilization:

$$U = \frac{\sum busy_time_i}{P \times M}$$

Speedup:

$$S = \frac{T_{sequential}}{T_{parallel}}$$

Load Imbalance:

$$LI = \frac{max_load - min_load}{average_load}$$

7 Datasets

This project uses synthetically generated task datasets.

Each task is characterized by:

- Execution time (random between 1 and 50 units)
- Dependencies on previous tasks

Experiments are conducted using:

- Task sizes: 50, 100, 200
- Number of processors: 2, 4, 8

This setup allows evaluation of scalability and performance under different workloads. The dataset is generated randomly to simulate real-world task scheduling scenarios.

The dependency probability is set to 20%, meaning each task has a 20% chance of depending on previous tasks. This ensures the formation of a realistic DAG structure.

Dataset Characteristics

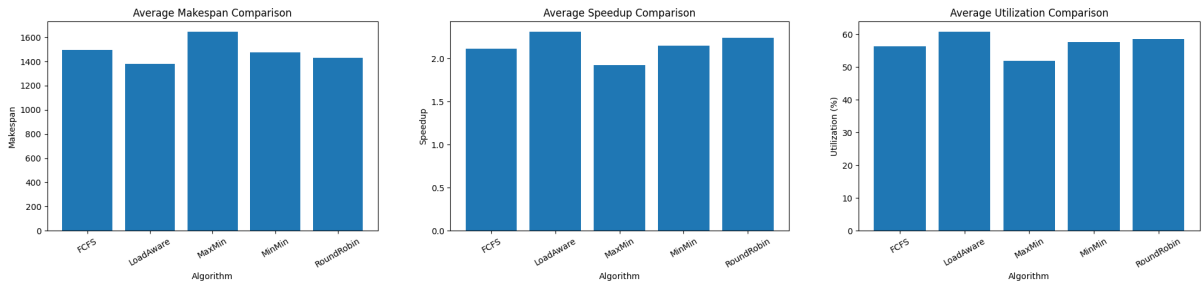
The dataset used in this project is synthetically generated to simulate realistic scheduling scenarios.

Each task is assigned a random execution time within a predefined range, ensuring variability in workload. Dependencies are introduced probabilistically to create a DAG structure that mimics real-world applications such as workflow systems and scientific computations.

The use of synthetic datasets allows controlled experimentation, enabling analysis of how different parameters affect scheduling performance. By varying task size and processor count, the scalability of algorithms can be evaluated effectively.

Multiple runs are conducted for each configuration to ensure consistency and reduce the impact of randomness. The results are averaged to provide reliable performance metrics.

8 Results and Discussion



(a) Makespan (b) Speedup (c) Utilization

Figure 3: Comparison of scheduling algorithms based on makespan, speedup, and utilization across multiple configurations

It is observed that Load-Aware achieves the lowest makespan and load imbalance, while Max-Min shows the worst performance. Round Robin maintains stable performance across all configurations.

8.1 Observations

The experiments were conducted under varying configurations:

- Task sizes: 50, 100, 200
- Processors: 2, 4, 8
- Execution time range: 1–50
- Dependency probability: 20%

Further, the experiments show that:

- Increasing processors from 2 to 8 reduces makespan but decreases utilization due to idle processors.
- Increasing task size increases makespan but improves utilization.
- Load imbalance is lowest in Load-Aware scheduling (as low as 0.02), while Max-Min shows highest imbalance (up to 3.7).

- Speedup increases with processors but is limited due to task dependencies.

Table 1: Best Performing Algorithm per Metric

Metric	Best Algorithm
Makespan	Load-Aware
Utilization	Load-Aware
Load Imbalance	Load-Aware
Speedup	Load-Aware / Round Robin

Interpretation of Results

The results clearly demonstrate that scheduling performance is highly dependent on both algorithm design and system configuration.

The Load-Aware algorithm consistently outperforms others due to its dynamic workload distribution strategy. By assigning tasks to the least loaded processor, it minimizes idle time and improves overall efficiency.

Round Robin shows stable performance across all configurations, indicating its robustness. However, it does not achieve optimal results due to lack of dependency awareness.

Min-Min performs well for smaller workloads but struggles as task size increases. This is because it tends to prioritize short tasks, leading to delays in executing longer tasks.

Max-Min exhibits the poorest performance, as prioritizing large tasks often results in underutilization of processors.

These findings highlight the importance of balancing multiple objectives rather than optimizing a single metric.

8.2 Detailed Analysis

From the experimental results, the following trends are observed:

- **Load-Aware Algorithm:** Consistently achieves the lowest makespan and lowest load imbalance. This is due to its dynamic load balancing strategy.
- **Round Robin:** Provides stable performance with moderate makespan and low imbalance due to equal distribution.

- **Min-Min:** Performs well for smaller tasks but suffers when larger tasks dominate.
- **Max-Min:** Shows the worst performance due to prioritization of large tasks, leading to idle processors.

Table 2: Average Performance Comparison

Algorithm	Makespan	Utilization (%)	Load Imbalance	Speedup
FCFS	1324	59.2	1.36	2.20
RoundRobin	1243	62.1	0.29	2.35
MinMin	1306	59.0	0.93	2.24
MaxMin	1435	54.2	2.45	2.05
LoadAware	1208	63.7	0.25	2.42

Time Complexity Analysis

The scheduling complexity depends on the number of tasks (n) and processors (p).

- FCFS: O(n)
- Round Robin: O(n)
- Min-Min / Max-Min: $O(n^2)$
- Load-Aware: O(n log p)

The Load-Aware algorithm provides a balance between efficiency and performance improvement.

Limitations

The study has the following limitations:

- Synthetic datasets may not fully represent real-world workloads
- The Load-Aware algorithm does not consider critical path optimization
- Communication cost between processors is not modeled
- Assumes homogeneous processors

9 Conclusion

The experimental results demonstrate that the Load-Aware algorithm consistently outperforms traditional scheduling techniques in terms of makespan, utilization, and load balancing.

The results also highlight that increasing the number of processors improves speedup but reduces utilization due to idle resources. Task dependencies significantly limit parallel performance, aligning with theoretical expectations such as Amdahlâs Law.

Overall, the study validates the effectiveness of load-balanced scheduling strategies in parallel computing systems.

Future Scope

Future work can extend this study in several directions.

One potential improvement is the implementation of dynamic scheduling algorithms that adapt to real-time system conditions. Additionally, incorporating machine learning techniques for predictive scheduling can further enhance performance.

The system can also be extended to distributed computing environments using frameworks such as MPI or cloud-based platforms. This would enable evaluation under more realistic conditions.

Another direction is to consider heterogeneous systems where processors have varying capabilities, introducing additional complexity to the scheduling problem.

Finally, integrating energy efficiency as a performance metric can provide a more comprehensive evaluation of scheduling algorithms.

References

[1] H. Topcuoglu, S. Hariri, and M. Wu, âPerformance-effective and low-complexity task scheduling for heterogeneous computing,â IEEE Transactions on Parallel and Distributed Systems, 2002.

[2] T. D. Braun et al., âA comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems,â Journal of Parallel and Distributed Computing, 2001.

[3] Y. Kwok and I. Ahmad, âStatic scheduling algorithms for allocating directed task graphs to multiprocessors,â ACM Computing Surveys, 1999.

[4] R. L. Graham, âBounds on multiprocessing timing anomalies,â SIAM Journal on Applied Mathematics, 1969.

[5] E. G. Coffman Jr., âComputer and Job-Shop Scheduling Theory,â Wiley, 1976.

[6] J. Blazewicz, K. Ecker, E. Pesch, G. Schmidt, and J. Weglarz, âScheduling in Computer and Manufacturing Systems,â Springer, 1994.

[7] O. Sinnen, âTask Scheduling for Parallel Systems,â Wiley-Interscience, 2007.

[8] I. Ahmad and Y. Kwok, âOn exploiting task duplication in parallel program scheduling,â IEEE Transactions on Parallel and Distributed Systems, 1998.

[9] K. Li, âScheduling precedence constrained tasks with reduced processor idle time,â IEEE Transactions on Parallel and Distributed Systems, 2012.

[10] J. Dongarra et al., âA set of level 3 basic linear algebra subprograms,â ACM Transactions on Mathematical Software, 1990.